

Exercises 32.3-1 to 32.3-5

String Matching with Finite Automata

Questions and solutions one by one

1 Exercise 32.3-1

Question 32.3-1

Construct the string-matching automaton for the pattern

$$P = \text{aabab}$$

and illustrate its operation on the text

$$T = \text{aaababaabaababaab}.$$

Solution

The pattern is

$$P = \text{aabab}, \quad m = 5.$$

So the states of the automaton are:

$$Q = \{0, 1, 2, 3, 4, 5\}.$$

State q means that the first q characters of the pattern currently match a suffix of the text scanned so far.

The only accepting state is:

$$\boxed{5}.$$

Step 1: Compute the transition function

We use:

$$\delta(q, a) = \sigma(P_q a),$$

where $\sigma(x)$ is the length of the longest prefix of P that is also a suffix of x .

Since the alphabet here is effectively

$$\Sigma = \{a, b\},$$

we compute $\delta(q, a)$ and $\delta(q, b)$ for each state.

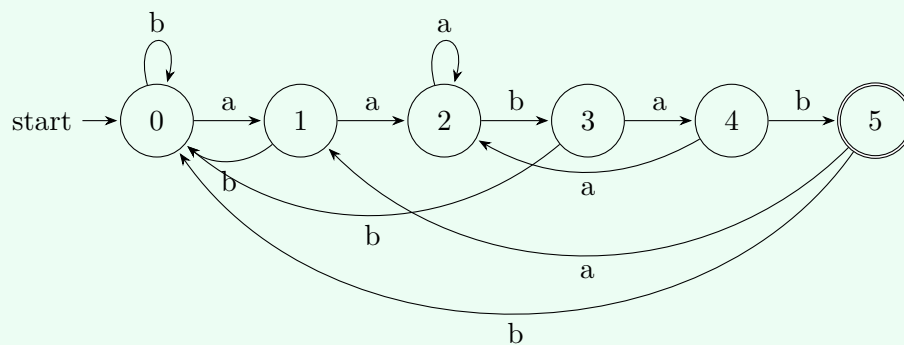
State q	$\delta(q, a)$	$\delta(q, b)$
0	1	0
1	2	0
2	2	3
3	4	0
4	2	5
5	1	0

Why these values?

A few examples:

- $\delta(0, a) = 1$: starting from nothing, reading a matches the first character.
- $\delta(2, b) = 3$: after matching aa , reading b gives suffix aab , so state 3.
- $\delta(4, b) = 5$: after matching $aaba$, reading b completes $aabab$.
- $\delta(5, a) = 1$: after a full match, reading a gives $aababa$; the longest prefix of P that is a suffix is only a , so the automaton returns to state 1.

State-transition diagram



Step 2: Run the automaton on the text

Text:

$T = \text{aaababaabaababab}$

We scan left to right.

i	$T[i]$	State after reading $T[i]$
1	a	1
2	a	2
3	a	2
4	b	3
5	a	4
6	b	5
7	a	1
8	a	2
9	b	3
10	a	4
11	a	2
12	b	3
13	a	4
14	b	5
15	a	1
16	a	2
17	b	3

Whenever the automaton reaches state 5, the pattern occurs.

So matches end at positions:

6, 14.

Therefore the shifts are:

$$6 - 5 = 1, \quad 14 - 5 = 9.$$

Final Answer

The string-matching automaton for $P = \text{aabab}$ has states 0 through 5, with transition table shown above. When run on

$$T = \text{aaababaabaababaab},$$

the automaton reaches the accepting state at positions

6, 14,

so the pattern occurs at shifts

1, 9.

2 Exercise 32.3-2

Question 32.3-2

Draw a state-transition diagram for a string-matching automaton for the pattern

$$P = \text{ababbabbababbababbabb}$$

over the alphabet

$$\Sigma = \{a, b\}.$$

Solution

We first note that

$$P = \text{ababbabbababbababbabb}$$

has length

$$m = 21.$$

Therefore the automaton has states

$$Q = \{0, 1, 2, \dots, 21\},$$

with start state 0 and accepting state

$$\boxed{21}.$$

Transition rule

For each state q and character $x \in \{a, b\}$,

$$\delta(q, x) = \sigma(P_q x),$$

where $\sigma(y)$ is the length of the longest prefix of P that is also a suffix of y .

Full transition table

The complete transition table is:

State q	$\delta(q, a)$	$\delta(q, b)$
0	1	0
1	1	2
2	3	0
3	1	4
4	3	5
5	6	0
6	1	7
7	3	8
8	9	0
9	1	10
10	11	0
11	1	12
12	3	13
13	14	0
14	1	15
15	16	8
16	1	17
17	3	18
18	19	0
19	1	20
20	3	21
21	9	0

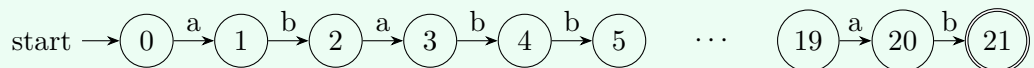
Main spine of the automaton

The forward matching edges follow the pattern exactly:

$$\begin{aligned}
& 0 \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{a} 3 \xrightarrow{b} 4 \xrightarrow{b} 5 \xrightarrow{a} 6 \xrightarrow{b} 7 \xrightarrow{b} 8 \xrightarrow{a} 9 \\
& \xrightarrow{b} 10 \xrightarrow{a} 11 \xrightarrow{b} 12 \xrightarrow{b} 13 \xrightarrow{a} 14 \xrightarrow{b} 15 \xrightarrow{b} 16 \xrightarrow{a} 17 \xrightarrow{b} 18 \xrightarrow{b} 19 \xrightarrow{a} 20 \xrightarrow{b} 21.
\end{aligned}$$

Compact state-transition diagram

Because the full diagram with all fallback edges is very wide, a compact diagram is shown below together with the exact transition table above.



How to read the fallback transitions

All non-spine transitions are obtained from the table. For example:

- $\delta(5, b) = 0$: from state 5, reading b destroys the current match and sends the automaton back to state 0.

- $\delta(15, b) = 8$: from state 15, reading b does not go to 0; instead it falls back to state 8, because the resulting suffix matches the first 8 characters of the pattern.
- $\delta(21, a) = 9$: after a full match, reading a leaves a suffix that matches the first 9 characters of the pattern, so the automaton continues in state 9.

Final Answer

Thus the required string-matching automaton for

$$P = \text{ababbabbababbababbabb}$$

over $\Sigma = \{a, b\}$ has:

- states 0 through 21,
- start state 0,
- accepting state 21,
- transition function exactly as given in the full table above.

That table completely determines the full state-transition diagram.

3 Exercise 32.3-3

Question 32.3-3

We call a pattern P **nonoverlappable** if $P_k \geq P_q$ implies $k = 0$ or $k = q$. Describe the state-transition diagram of the string-matching automaton for a nonoverlappable pattern.

Solution

A pattern is **nonoverlappable** if no proper prefix of the pattern is also a suffix of any prefix of the pattern, except the trivial empty prefix.

That means:

$$P_k \geq P_q \implies k = 0 \text{ or } k = q.$$

What this means for the automaton

Normally, after a mismatch, the automaton may fall back to some smaller state $k > 0$. But for a nonoverlappable pattern, there are no useful proper prefix-suffix overlaps. So after a mismatch, the automaton almost always falls back directly to state 0.

Structure of the automaton

Suppose the pattern length is m . Then:

- states are $0, 1, 2, \dots, m$,
- the main spine is:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow m,$$

following the characters of the pattern,

- every mismatch transition from state q goes to state 0,
- except possibly when the mismatch character equals the first character of the pattern, in which case the automaton may go to state 1.

Why?

Since there is no nontrivial overlap, once the current match fails, the matched suffix cannot help continue the next match.

So the machine has no meaningful fallback state other than:

0

or occasionally:

1

if the current input character can begin a fresh match.

Diagram description

The automaton looks like a simple straight chain:

$$0 \xrightarrow{P[1]} 1 \xrightarrow{P[2]} 2 \xrightarrow{P[3]} \dots \xrightarrow{P[m]} m$$

and almost all other transitions go back to 0.

Key Shape

For a nonoverlappable pattern, the automaton is almost a straight line with reset edges back to 0.

Final Answer

For a nonoverlappable pattern, the state-transition diagram consists of:

- a main spine $0 \rightarrow 1 \rightarrow \dots \rightarrow m$ for correct matching characters,
- mismatch transitions that reset directly to state 0,
- with possible transitions to state 1 when the mismatch character equals the first character of the pattern.

So the automaton is much simpler than in the overlapping case.

4 Exercise 32.3-4

Question 32.3-4

Given two patterns P and P' , describe how to construct a finite automaton that determines all occurrences of either pattern. Try to minimize the number of states in your automaton.

Solution

We want one automaton that accepts whenever the text ends with:

$$P \quad \text{or} \quad P'.$$

Basic idea

Instead of building two completely separate automata, we combine them.

A state should represent:

the longest suffix of the text scanned so far that is also a prefix of either P or P' .

So we should not keep duplicate states for prefixes that are identical in the two patterns.

Best construction idea: use merged prefix states

Collect all prefixes of P and all prefixes of P' , including the empty string.

Then merge equal prefixes into a single state.

So each state corresponds to a distinct string x such that:

$$x \text{ is a prefix of } P \text{ or } P'.$$

The accepting states are:

$$P \quad \text{and} \quad P'.$$

Transition rule

If the current state corresponds to string x , and the next input character is a , then move to the longest suffix of xa that is also a prefix of P or P' .

This is exactly the same idea as ordinary string-matching automata, except now the allowed prefixes come from either pattern.

Why this minimizes the number of states

If P and P' share common prefixes, we should reuse them instead of duplicating them.

For example, if:

$$P = \text{abab}, \quad P' = \text{abac},$$

then the prefixes

$$\varepsilon, \text{a}, \text{ab}, \text{aba}$$

are shared.

So a merged automaton is smaller than two separate automata.

Conceptual structure

This construction is equivalent to building a trie of the two patterns and then adding fallback transitions based on longest suffix-prefix matches.

Minimal-State Idea

Merge all common prefixes of the two patterns into shared states.

Final Answer

To construct a finite automaton for matching either P or P' :

1. Create one state for every distinct prefix of P or P' .
2. Merge identical prefixes into the same state.
3. Mark the states corresponding to P and P' as accepting.
4. For each state x and character a , transition to the longest suffix of xa that is also a prefix of P or P' .

This construction minimizes states by sharing all common prefixes.

5 Exercise 32.3-5

Question 32.3-5

Given a pattern P containing gap characters, show how to build a finite automaton that can find an occurrence of P in a text T in $O(n)$ matching time, where $n = |T|$.

Solution

Let the pattern contain a special gap symbol that can match any string, including the empty string.

For example:

$$P = X_1 \text{ gap } X_2 \text{ gap } \cdots \text{ gap } X_k,$$

where each X_i is an ordinary string without gap symbols.

Main idea

A match occurs if the text contains:

$$X_1 \text{ followed later by } X_2 \text{ followed later by } \cdots \text{ followed later by } X_k.$$

The gap can absorb any substring between two consecutive fixed blocks.

So the automaton must keep track of:

how many blocks have already been matched in order, and how much of the current block is matched so far.

Construction idea

For each block X_i , build the usual string-matching automaton for that block.

Then connect these automata in sequence:

- once the automaton for X_1 reaches its accepting state, the machine moves into the automaton for X_2 ,
- similarly from X_2 to X_3 , and so on.

Because the gap can match any substring, once block X_i is completed, the automaton remains ready to begin matching block X_{i+1} at any later position.

State meaning

A state can be represented as:

$$(i, q),$$

where:

- i tells which block we are currently matching,
- q tells how many characters of that block have been matched.

Once the last block X_k is fully matched, we accept.

Why matching time is $O(n)$

As the text is scanned left to right:

- each character causes one transition,
- the automaton never backs up in the text,
- all state changes are constant-time transitions.

So after preprocessing, the matching phase takes:

$$O(n).$$

Preprocessing

The preprocessing builds a finite automaton whose states encode:

- current block number,
- current matched prefix length inside that block.

This preprocessing depends only on the pattern, not on the text.

Final Answer

To handle gap characters in a finite automaton:

1. split the pattern into ordinary blocks

$$X_1, X_2, \dots, X_k,$$

2. build automaton states that track which block is active and how much of it is matched,
3. after completing one block, allow arbitrary text until the next block begins matching,
4. accept when the final block is completed.

Since the resulting automaton scans the text once from left to right, the matching time is

$$O(n).$$